Listing 10.4 Add `famous_for` to the Promoter class

```
# RiffMates/promoters/models.py
from django.db import models

class Promoter(models.Model):
    first_name = models.CharField(max_length=25)
    last_name = models.CharField(max_length=25)
    famous_for = models.CharField(max_length=50)
```

A new field to store what a promoter is known for

The purpose of migrations is to manage your database over time as your project changes. Once you've released code and your project is live, you'll have data in your database, which makes changing it messier than creating it. In `Promoter`, the first and last name are required fields, since you haven't specified `null=True`. The database expects all rows in the table to contain a value. First and last name are both text, which means the value can be an empty string. From the database's perspective, an empty string isn't empty, `NULL` is empty.

> **NOTE** Recall `null=True` specifies a value can be `NULL` in the database, while `blank=True` specifies whether the Django Admin and other forms allow you to enter an empty string.

The new `famous_for` field is also required. When the new column gets added to the table, the database won't allow the `famous_for` field on the existing rows to be `NULL`. You have to put something in each field, for the sake of database integrity. Your choices are to change `famous_for`, so it isn't a required field; provide a default value on the field; or provide a default value in the migration file.

When you run `makemigrations` on the changed `Promoter` model, Django detects that `famous_for` is a required field without a default value and prompts for content to insert during the migration. This wasn't needed when you created the table, as there was no content in the database, but there is now. Run the `makemigrations` command as follows, choosing Provide a One-Off Default when prompted:

```
(venv) RiffMates$ python manage.py makemigrations promoters
It is impossible to add a non-nullable field 'famous_for' to promoters
without specifying a default. This is because the database needs
something to populate existing rows.
Please select a fix:
 1) Provide a one-off default now (will be set on all existing rows
 with a null value for this column)
 2) Quit and manually define a default value in models.py.
Select an option:
```

When you select Provide a One-Off Default, Django opens a Python REPL for you where you enter the default value. The use of the shell here is rather clever; it means you can call functions that return a value. Note that if you do this, it is called immediately, and the value gets inserted in the script. The calculation isn't performed at migration time or per row in the database.